МИНОБРНАУКИ РОССИИ

Воронежский государственный университет
Факультет прикладной математики, информатики и механики
Кафедра математического обеспечения ЭВМ

Цифровая обработка изображений

Лабораторная работа № 2

Выделение контуров и пространственная фильтрация
Варианты 1, 2 и 3

Выполнил(а): _____

Группа: _____

Преподаватель: Сырых А. С.

Воронеж, 2026

1. Постановка задачи

Вариант 1. Выделение контуров на изображении операторами Лапласа, Робертса и Собела. Вариант 2. Удаление гауссова шума в пространственной области усредняющими фильтрами. Вариант 3. Удаление гауссова шума порядковыми фильтрами.

Требуется реализовать программу на C++, подготовить изображения, демонстрирующие различия методов, и проанализировать результаты. Цвет не является предметом работы, поэтому обработка выполняется в градациях серого. Все варианты представлены в одном проекте и одном отчёте.

Дополнительно реализуется заданный класс Image: чтение и запись PNG, поддержка 1–4 каналов, работа с пикселями, глубокое копирование и безопасное поведение пустого объекта. Интерфейс Image.h сохранён без добавления методов.

2. Математическая модель

Пусть $f(x,y)$ — яркость пикселя, $0 \le f \le 255$. Цветной вход преобразуется в полутоновый по формуле (1); для одно- и двухканального входа используется непосредственно канал яркости.

$$f = \mathrm{round}(0.299\,R + 0.587\,G + 0.114\,B). \quad (1)$$

Все окрестности за пределами растра продолжаются ближайшим граничным значением: координаты ограничиваются диапазонами $[0,W-1]$ и $[0,H-1]$. Это предотвращает появление ложного контура на постоянном фоне у края изображения.

Итоговые значения округляются до ближайшего целого и ограничиваются диапазоном байта. Ограничение амплитуды не является нормализацией по максимуму конкретного изображения:

$$Q(t) = \mathrm{round}(\min(255, \max(0, t))). \quad (2)$$

2.1. Операторы выделения контуров

Для Лапласа используется четырёхсвязная дискретная вторая производная (3). Модуль позволяет отобразить положительные и отрицательные отклики одной яркостью.

$$L = f(x-1,y)+f(x+1,y)+f(x,y-1)+f(x,y+1)-4f(x,y);$$

$$g(x,y) = Q(|L|). \quad (3)$$

Робертс вычисляет две диагональные разности в окне 2×2. Результат определяется длиной вектора (4). Малое окно обеспечивает компактный отклик, но не содержит усреднения шума.

$$a = f(x,y)-f(x+1,y+1);\ b = f(x+1,y)-f(x,y+1);$$

$$g(x,y) = Q(\sqrt{(a^2+b^2)}). \quad (4)$$

2.2. Собел и фильтры подавления шума

Собел использует две маски 3×3 (5). Каждая совмещает разность по одной координате и усреднение по другой. Суммирование выполняется по соответствующим смещениям маски.

$$Sx = [-1\ 0\ 1;\ -2\ 0\ 2;\ -1\ 0\ 1];$$

$$Sy = [-1\ -2\ -1;\ 0\ 0\ 0;\ 1\ 2\ 1];$$

$$Gx = \Sigma_i\Sigma_j\ Sx[j,i]\ f(x+i,y+j),\ \ i,j = -1,0,1;$$

$$Gy = \Sigma_i\Sigma_j\ Sy[j,i]\ f(x+i,y+j);$$

$$g(x,y) = Q(\sqrt{(Gx^2+Gy^2)}). \quad (5)$$

Порядок для всех трёх операторов: получить яркостное изображение; вычислить разности по (3), (4) или (5) для каждого пикселя; применить (2); записать результат в новый объект Image. Исходное изображение не изменяется.

При арифметическом усреднении (6) все k^2 пикселей окна имеют одинаковый вес. В работе сравниваются k=3 и k=5; радиус окна r=(k−1)/2.

$$g(x,y) = Q((1/k^2)\ \Sigma_i\ \Sigma_j\ f(x+i,y+j)),\ i,j = -r,\ldots,r. \quad (6)$$

Гауссово усреднение (7) придаёт больший вес близким пикселям. Постоянный множитель гауссовой функции сокращается при нормировании. Используется окно 5×5, σ=1.0.

$$v(i,j) = \exp(-(i^2+j^2)/(2\sigma^2));\ w(i,j) = v(i,j)/\Sigma_i\Sigma_j v(i,j);$$

$$g(x,y) = Q(\Sigma_i\Sigma_j\ w(i,j)\ f(x+i,y+j)). \quad (7)$$

Алгоритм усреднения: сформировать и нормировать веса; для каждого пикселя суммировать взвешенные значения окна; округлить результат по (2).

При σ=0 программа выбирает равные веса (6), не подставляя ноль в гауссову формулу.

Для порядковых фильтров составляется набор значений окна, расположенный по возрастанию $z_1 \le \dots \le z_n$, где $n=k^2$. Тогда (8) определяет три разные операции:

$$g_min = z_1;\ g_med = z[(n+1)/2];\ g_max = z_n. \quad (8)$$

Минимум сужает светлые области, максимум расширяет их. Медиана выбирает центральное значение и менее чувствительна к отдельным выбросам, однако также способна удалять мелкие детали. В программе медиана находится через std::nth_element без полной сортировки.

Для численного сравнения с чистым эталоном используются средняя квадратичная ошибка MSE и PSNR (9). Меньшая MSE и больший PSNR соответствуют лучшему совпадению яркостей, но не заменяют оценку сохранности деталей.

$$MSE = (1/(WH))\ \Sigma_x\ \Sigma_y\ (g(x,y)-f(x,y))^2;$$

$$PSNR = 10\ \log_{10}(255^2/MSE),\ дБ. \quad (9)$$

3. Программная реализация и проверка

Реализация написана на C++17. Внешние библиотеки из задания используются только для PNG (stb_image, stb_image_write) и тестов (doctest). Готовые функции фильтрации сторонних библиотек не используются.

Файл / функция	Назначение
Image.h, Image.cpp	Заданный интерфейс и работа с памятью, пикселями и PNG.
Processing.h, Processing.cpp	Отдельные функции обработки без расширения Image.
processing::grayscale	Преобразование яркости по (1).
processing::edges	Выбор Лапласа, Робертса или Собела; формулы (3)–(5).

Файл / функция	Назначение
processing::average	Равные или гауссовы веса; формулы (6), (7).
processing::rank	Минимум, медиана и максимум; формула (8).
main.cpp	Чтение пути входа и каталога выхода, последовательный запуск всех методов.

Данные Image хранятся в std::unique_ptr<uint8_t[]>. Копирование выделяет отдельную память. Для 1–2 каналов getColor возвращает яркость в r, а g и b устанавливает в ноль согласно смыслу предоставленных тестов. Для 3–4 каналов возвращается RGB. setColor сохраняет альфа-канал; при создании двух- и четырёхканального изображения он равен 255. При недопустимых координатах объект и выходные аргументы не меняются.

Сборка и запуск из корня проекта: make; make test; make demo. Для своего файла: ./build/lab input.png output/my_image. Программа сохраняет 11 PNG: полутоновый вход, три результата выделения контуров, три усреднения и четыре порядковых фильтра. Ошибка чтения или записи приводит к ненулевому коду завершения.

В текущей проверке выполнено 18 тестовых случаев, 474 проверки; ошибок и пропущенных тестов нет. Из них 12 случаев взяты из задания и активированы, ещё 6 проверяют дополнительные свойства. Тесты и приложение собираются отдельно, поэтому конфликт двух main исключён.

Дополнительные проверки: нулевой отклик контурных операторов на постоянном поле; сохранение его яркости всеми фильтрами, включая края; ручной расчёт отклика на ступень; результат обработки одиночного импульса; сохранение альфа-канала после PNG-записи; независимость присваиваемой копии; отклонение некорректных размеров окна.

Для ступени 0→40 проверены отклики Лапласа 40, Робертса round(40√2)=57 и Собела 160. Для импульса 255 среднее 3×3 даёт 28,

гауссово окно 3×3 при σ=1 даёт 52, медиана и минимум — 0. Такие проверки связывают реализацию с математическими формулами, а не только с успешным запуском.

4. Тестовые изображения

Подготовлены две самостоятельные сцены 256×192 пикселя. Набор А содержит прямоугольник, круг, треугольник, крупные однородные области и участок линейного изменения яркости. Набор Б содержит тонкие горизонтальные и вертикальные штрихи, шахматный фрагмент, окружность и диагональ на градиентном фоне. Первый набор удобен для оценки шума на ровном фоне, второй — для обнаружения потери деталей.

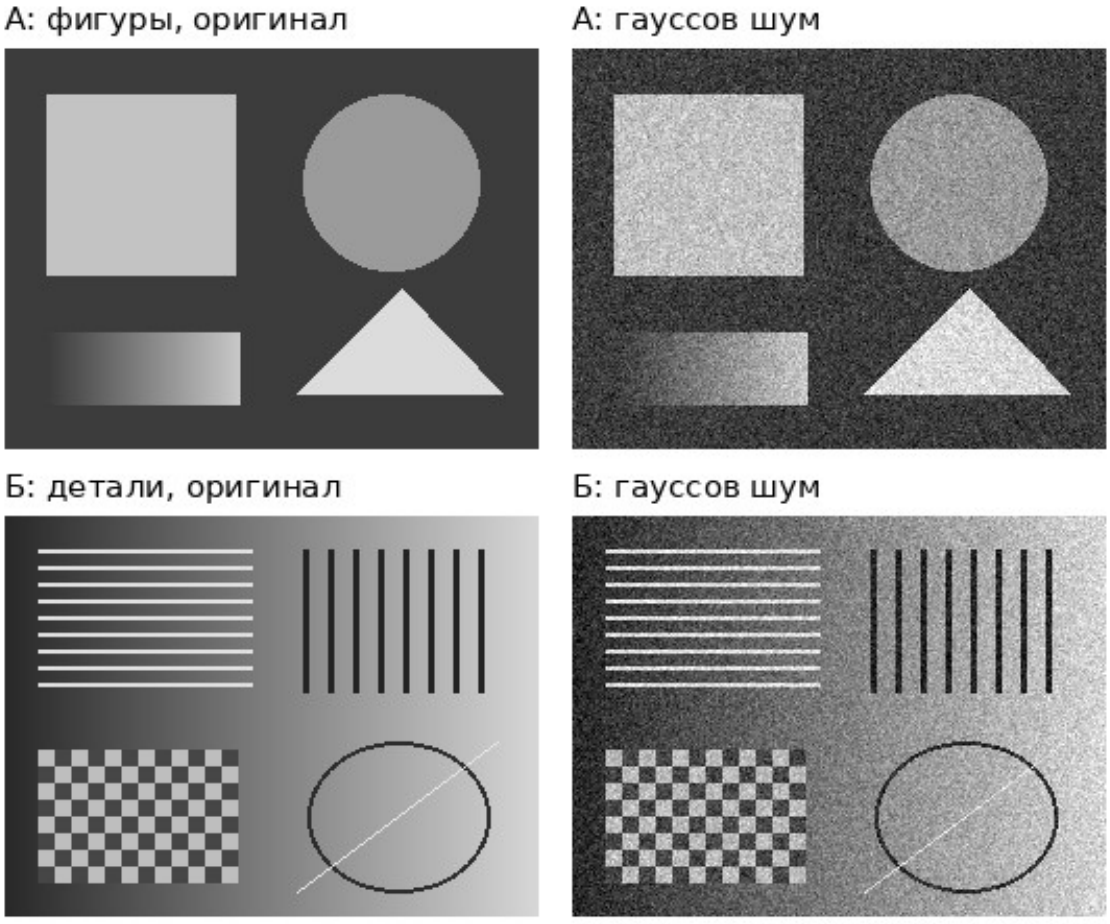


Рисунок 1. Чистые эталоны (слева) и соответствующие изображения с гауссовым шумом (справа).

Шум добавлен вне программы фильтрации редактором ImageMagick: -seed 731 (А) / 1201 (Б), -attenuate 0.9 +noise Gaussian, глубина 8 бит. После

добавления результат сохранён в PNG. Тестовые сцены подготовлены специально для сравнения этих методов.

Измеренное стандартное отклонение разности с эталоном составляет 17.92 для А и 17.92 для Б. Средняя разность равна -0.60 и -0.65; небольшое смещение связано в том числе с дискретизацией. Значения после добавления шума ограничиваются диапазоном 0–255.

Для обоих вариантов подавления шума используются одинаковые зашумлённые входы. Чистые эталоны сохранены отдельно, поэтому ошибка по (9) вычисляется относительно известного исходного сигнала.

5. Вариант 1: выделение контуров

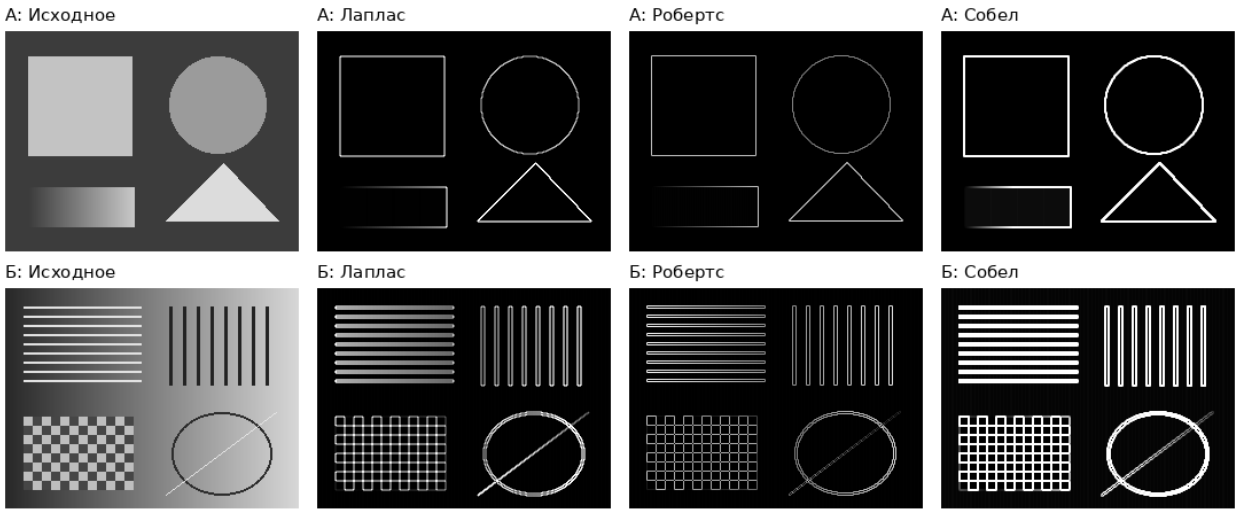


Рисунок 2. Сверху набор А, снизу набор Б. В каждом ряду: вход, Лаплас, Робертс, Собел.

На наборе А Лаплас выделяет обе стороны скачка яркости, поскольку соседние значения второй разности имеют разные знаки, а отображается их модуль (3). Робертс даёт компактный отклик на границах фигур и реагирует на диагональные перепады. Собел формирует более широкий яркий контур из-за окна 3×3 и коэффициентов ±2.

Набор Б показывает, что тонкие штрихи создают близко расположенные отклики; при пересечении или сближении линий они могут сливаться. На линейном градиенте внутренняя вторая разность Лапласа близка к нулю, а операторы первого порядка реагируют на изменение

яркости. В местах целочисленного округления возможен слабый остаточный отклик.

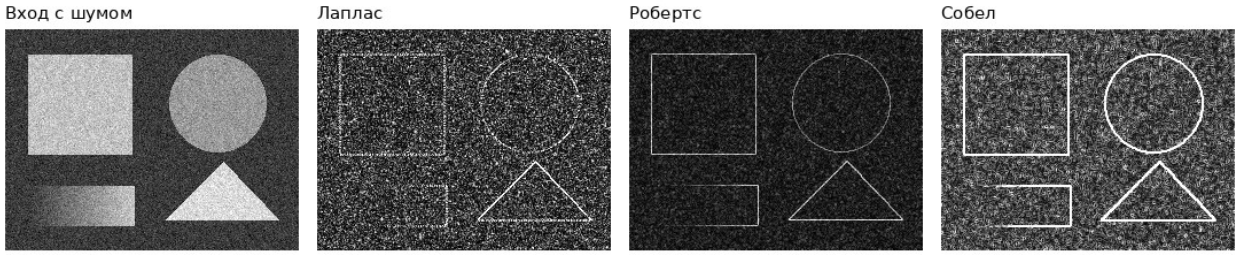


Рисунок 3. Набор A с шумом и результаты тех же трёх операторов.

На зашумлённом входе появляются отклики внутри однородных областей: дифференцирование усиливает локальные колебания. У Собела есть сглаживание в поперечном направлении, однако его ненормированные маски также усиливают амплитуду. Сравнивать только яркость шумового фона у трёх выходов некорректно, поскольку коэффициенты операторов различаются. Во всех случаях изображения по рисунку 2 позволяют отделить полезную реакцию на границу от реакции на шум.

При сильных перепадах результат ограничивается 255 по (2). Поэтому яркость контура после насыщения уже не пропорциональна точной величине производной. Предварительное подавление шума могло бы уменьшить ложные отклики, но в этом сравнении операторы применены непосредственно ко входу, чтобы показать их собственные свойства.

6. Вариант 2: усредняющие фильтры

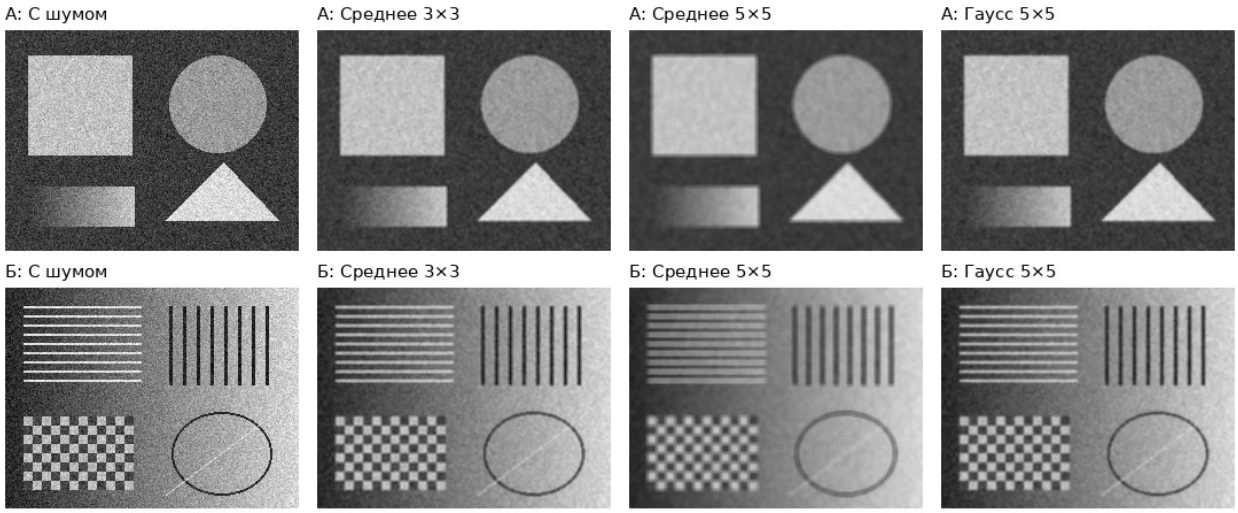


Рисунок 4. Сверху набор А, снизу набор Б. Вход с шумом; среднее 3×3; среднее 5×5; Гаусс 5×5, σ=1.

Метод	А: MSE	А: PSNR, дБ	Б: MSE	Б: PSNR, дБ
Вход с шумом	321.50	23.06	321.63	23.06
Среднее 3×3	111.66	27.65	437.16	21.72
Среднее 5×5	147.53	26.44	873.40	18.72
Гаусс 5×5	95.38	28.34	400.27	22.11

На крупных фигурах все три фильтра уменьшают ошибку относительно зашумлённого входа. Из усредняющих методов лучшие значения на обоих наборах получены для гауссова окна 5×5 с σ=1. Более низкие веса удалённых пикселей снижают смешивание яркостей по разные стороны границы по сравнению с равными весами окна 5×5.

Увеличение арифметического окна с 3×3 до 5×5 сильнее сглаживает фон, но не обязательно улучшает результат целиком: MSE на А возрастает с 111.66 до 147.53 из-за размытия краёв. На Б тонкие штрихи имеют ширину, сопоставимую с окном, поэтому усреднение смешивает их с фоном. Ошибка всех трёх усредняющих методов на этом наборе оказывается выше, чем у входа с шумом.

Такой результат согласуется с (6), (7): усредняется не только шум, но и полезное изменение яркости. Шум на однородных участках ослабляется,

однако общий PSNR может ухудшиться из-за утраты мелких объектов. Следовательно, размер окна следует выбирать с учётом масштаба деталей, а не только видимой гладкости изображения.

7. Вариант 3: порядковые фильтры

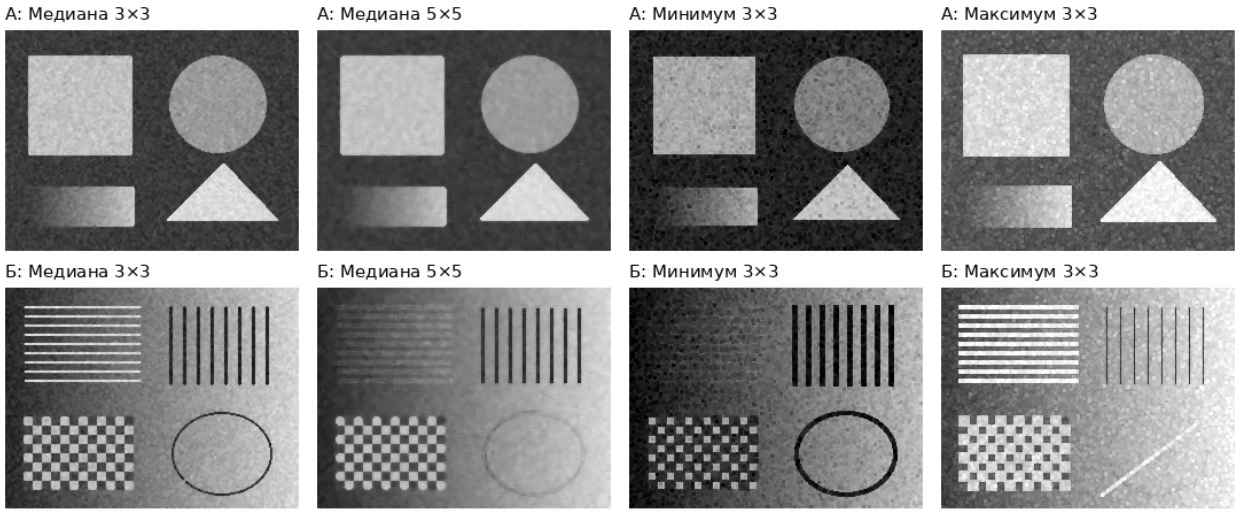


Рисунок 5. Сверху набор А, снизу набор Б. Медиана 3×3; медиана 5×5; минимум 3×3; максимум 3×3.

Метод	А: MSE	А: PSNR, дБ	Б: MSE	Б: PSNR, дБ
Вход с шумом	321.50	23.06	321.63	23.06
Медиана 3×3	65.27	29.98	124.90	27.17
Медиана 5×5	44.62	31.64	830.54	18.94
Минимум 3×3	1306.65	16.97	2975.02	13.40
Максимум 3×3	1230.13	17.23	2866.88	13.56

На наборе А медиана 5×5 даёт наименьшую ошибку среди проверенных методов: MSE=44.62. Большинство элементов окна на широком однородном участке относятся к одному уровню яркости; медиана подавляет колебания без арифметического смешивания яркостей всех элементов. Однако около углов и малых фигур граница изменяется.

На наборе Б медиана 3×3 сохраняет больше деталей и даёт MSE=124.90. Для окна 5×5 ошибка возрастает до 830.54: тонкая светлая

линия может занимать меньше половины окна, поэтому медиана заменяет её фоном. Таким образом, выигрыш большого окна на первом наборе не переносится на второй.

Минимум и максимум не являются универсальными средствами устранения гауссова шума. Первый выбирает преимущественно отрицательные отклонения и затемняет изображение; второй выбирает положительные и осветляет его. Среднее смещение относительно эталона для А равно −29.71 и +28.41, для Б — −38.96 и +37.80 соответственно. При этом светлые детали сужаются после минимума и расширяются после максимума, что прямо следует из (8).

На данных двух сценах медианные фильтры с подходящим размером окна превосходят проверенные усредняющие методы по MSE. Это наблюдение относится к выбранным изображениям и уровню шума; оно не устанавливает преимущество медианы для любого гауссова шума или любого содержания изображения.

8. Вывод

Реализован заданный класс Image с неизменённым интерфейсом и единая программа для всех трёх вариантов. Операторы Лапласа, Робертса и Собела выделяют разные по форме и ширине отклики на границы. Непосредственная обработка зашумлённого входа показывает появление ложных контуров.

Усредняющие фильтры уменьшают колебания на равномерном фоне, но размывают мелкие объекты. Гауссово окно 5×5 с $\sigma=1$ показало меньшую ошибку, чем два рассмотренных арифметических окна. Медиана 5×5 эффективна на крупных фигурах, а на изображении с тонкими деталями лучше сработало окно 3×3. Минимум и максимум меняют геометрию светлых областей и дают значительное смещение яркости.

Все 18 тестовых случаев пройдены. Демонстрационный запуск обработал четыре входных PNG и сохранил 44 выходных файла. Выводы

основаны на полученных изображениях и сравнении с чистыми эталонами; заранее заданного универсального победителя среди методов нет.

Источники

1. Материалы задания: lab_1.pdf (внутри — «Лабораторная работа №2»), image_env.pdf, ReadMe.pdf, исходный интерфейс Image и tests_image.cpp.

2. Библиотеки stb_image и stb_image_write для чтения и записи PNG. Заголовочные файлы предоставлены вместе с заданием.

3. Библиотека doctest. Тестовое окружение и проверки класса Image из материалов задания.

4. Справочная документация ImageMagick. Добавление гауссова шума, выбор его интенсивности и начального значения генератора.

Приложения

В приложении А приведён модуль обработки: вспомогательные функции, преобразование в оттенки серого и реализации всех трёх вариантов. В приложении Б приведена реализация класса Image. Точка входа и тесты находятся в проекте; сторонние заголовки в текст отчёта не включены.

Приложение А. Алгоритмы обработки

```
#include "Processing.h"
#include <algorithm>
#include <cmath>
#include <vector>

namespace {
void requireGray(const Image& image) {
    if (image.width() <= 0 || image.height() <= 0 || image.nChannels() != 1)
        throw std::invalid_argument("Expected a nonempty grayscale image");
}
void requireWindow(int window) {
    if (window < 1 || window > 31 || window % 2 == 0)
        throw std::invalid_argument("Window must be odd and between 1 and 31");
}
int sample(const Image& image, int x, int y) {
    // Replicate the closest border pixel; do not add artificial black edges.
    x = std::clamp(x, 0, image.width() - 1);
    y = std::clamp(y, 0, image.height() - 1);
    int r = 0, g = 0, b = 0;
    image.getColor(x, y, r, g, b);
    return r;
}
void put(Image& image, int x, int y, double value) {
    const int v = static_cast<int>(std::lround(std::clamp(value, 0.0, 255.0)));
    image.setColor(x, y, v, v, v);
}
}

namespace processing {
Image grayscale(const Image& input) {
    if (input.width() <= 0 || input.height() <= 0)
        throw std::invalid_argument("Empty input image");
    Image result(input.width(), input.height(), 1);
    for (int y = 0; y < input.height(); ++y)
        for (int x = 0; x < input.width(); ++x) {
            int r = 0, g = 0, b = 0;
            input.getColor(x, y, r, g, b);
            put(result, x, y, input.nChannels() < 3 ? r : 0.299*r + 0.587*g + 0.114*b);
        }
    return result;
}

Image edges(const Image& gray, Edge method) {
    requireGray(gray);
```

```
                Image    result(gray.width(),    gray.height(),    1);
            const   int   sx[3][3]   =   {{-1,0,1},   {-2,0,2},   {-1,0,1}};
            const   int   sy[3][3]   =   {{-1,-2,-1},   {0,0,0},   {1,2,1}};
               for   (int   y   =   0;   y   <   gray.height();   ++y)
                     for   (int   x   =   0;   x   <   gray.width();   ++x)   {
                                                 double    value    =    0;
                                    if   (method   ==   Edge::Laplace)   {
                                  value   =   std::abs(sample(gray,x-1,y)   +
sample(gray,x+1,y)                                                          +
                                                      sample(gray,x,y-1)   +
sample(gray,x,y+1)                    -                4*sample(gray,x,y));
                               }   else   if   (method   ==   Edge::Roberts)   {
                  const int a = sample(gray,x,y) - sample(gray,x+1,y+1);
                  const int b = sample(gray,x+1,y) - sample(gray,x,y+1);
                                          value   =   std::hypot(a,   b);
                                                        }    else    {
                                             int   gx   =   0,   gy   =   0;
                                          for   (int   j   =   -1;   j   <=   1;   ++j)
                                          for   (int   i   =   -1;   i   <=   1;   ++i)   {
                                           const   int   p   =   sample(gray,x+i,y+j);
                                                     gx   +=   p   *   sx[j+1][i+1];
                                                     gy   +=   p   *   sy[j+1][i+1];
                     }
```

```
                                        value = std::hypot(gx, gy);
                                                                }
                                        put(result, x, y, value);
                                                                }
                                                return result;
}

Image average(const Image& gray, int window, double sigma) {
                                                requireGray(gray);
                                              requireWindow(window);
                if (!std::isfinite(sigma) || sigma < 0)
              throw std::invalid_argument("Sigma must be finite and
nonnegative");
                    const int radius = window / 2;
             std::vector<double> weights(window * window, 1.0);
                                double total = 0;
              for (int j = -radius; j <= radius; ++j)
                   for (int i = -radius; i <= radius; ++i) {
                     double& w = weights[(j+radius)*window+i+radius];
              if (sigma > 0) w = std::exp(-(i*i+j*j)/(2*sigma*sigma));
                                                total += w;
                                                                }
               for (double& w : weights) w /= total;
                 Image result(gray.width(), gray.height(), 1);
              for (int y = 0; y < gray.height(); ++y)
                   for (int x = 0; x < gray.width(); ++x) {
                                              double sum = 0;
                        for (int j = -radius; j <= radius; ++j)
                          for (int i = -radius; i <= radius; ++i)
                          sum += weights[(j+radius)*window+i+radius] *
sample(gray,x+i,y+j);
                                                put(result,x,y,sum);
                                                                }
                                            return result;
}

Image rank(const Image& gray, int window, Rank method) {
                                                requireGray(gray);
                                              requireWindow(window);
                Image result(gray.width(), gray.height(), 1);
                  const int radius = window / 2;
                  std::vector<int> values(window * window);
              for (int y = 0; y < gray.height(); ++y)
                   for (int x = 0; x < gray.width(); ++x) {
                                                  int k = 0;
                        for (int j = -radius; j <= radius; ++j)
                          for (int i = -radius; i <= radius; ++i)
                                  values[k++] = sample(gray,x+i,y+j);
                                                int value;
                                    if (method == Rank::Minimum)
```

```
                                value  =  *std::min_element(values.begin(),
values.end());
                                   else   if   (method   ==   Rank::Maximum)
                                value  =  *std::max_element(values.begin(),
values.end());
                                                                      else     {
                             auto  middle  =  values.begin()  +  values.size()/2;
                                     std::nth_element(values.begin(),  middle,
values.end());
                                                                value   =   *middle;
                                                                             }
                                                              put(result,x,y,value);
                                                                             }
                                                          return            result;
}
}
```

Приложение Б. Класс Image

```
#define                                          STB_IMAGE_IMPLEMENTATION
#define                                    STB_IMAGE_WRITE_IMPLEMENTATION
#include                                                    "stb_image.h"
#include                                                "stb_image_write.h"
#include                                                         "Image.h"
#include                                                         <cstring>
#include                                                          <limits>

Image::Image()                                =                          default;

Image::Image(int    width,    int    height,    int    nChannels)    {
    if (width <= 0 || height <= 0 || nChannels < 1 || nChannels > 4)
            throw  std::invalid_argument("Invalid  image  dimensions  or
channel                                                              count");
         const  size_t  limit  =  std::numeric_limits<size_t>::max();
     if (static_cast<size_t>(width) > limit / height / nChannels ||
             width  >  std::numeric_limits<int>::max()  /  nChannels)
                 throw  std::length_error("Image  is  too  large");
                                      m_width          =          width;
                                    m_height          =         height;
                                m_nChannels         =        nChannels;
       m_size  =  static_cast<size_t>(width)  *  height  *  nChannels;
                m_data    =    std::make_unique<uint8_t[]>(m_size);
             if    (nChannels    ==    2    ||    nChannels    ==    4)
         for (size_t i = nChannels - 1; i < m_size; i += nChannels)
                                                 m_data[i]    =    255;
}

Image::Image(const          std::string&          filepath)          {
    if (!isRead(filepath)) throw std::runtime_error("Cannot read: " +
filepath);
}

Image::Image(const                      Image&                      other)
             :    m_width(other.m_width),    m_height(other.m_height),
            m_nChannels(other.m_nChannels),  m_size(other.m_size)  {
                                     if          (m_size)          {
                    m_data   =   std::make_unique<uint8_t[]>(m_size);
             std::memcpy(m_data.get(),  other.m_data.get(),  m_size);
                                                                    }
}

Image&       Image::operator=(const       Image&       other)       {
                   if    (this    ==    &other)    return    *this;
       //  Allocate  before  changing  the  object:  allocation  failure
preserves                             its                            state.
                        auto      copy      =      other.m_size      ?
std::make_unique<uint8_t[]>(other.m_size)          :          nullptr;
            if   (copy)   std::memcpy(copy.get(),   other.m_data.get(),
other.m_size);
```

```
                                m_data        =        std::move(copy);
                                 m_width        =        other.m_width;
                                m_height        =        other.m_height;
                              m_nChannels       =       other.m_nChannels;
                                     m_size         =         other.m_size;
                                                  return            *this;
}

bool       Image::isRead(const       std::string&       filepath)       {
               int    w    =    0,    h    =    0,    channels    =    0;
           std::unique_ptr<stbi_uc,   decltype(&stbi_image_free)>   raw(
                 stbi_load(filepath.c_str(),  &w,  &h,  &channels,  0),
stbi_image_free);
                             if       (!raw)       return       false;
                          Image      loaded(w,      h,      channels);
         std::memcpy(loaded.m_data.get(),  raw.get(),  loaded.m_size);
                                       *this          =          loaded;
                                                 return            true;
}

bool Image::isWrite(const std::string& filepath) const {
```

Приложение Б. Класс Image (продолжение)

```
                            if      (!m_data)      return      false;
         return  stbi_write_png(filepath.c_str(),  m_width,  m_height,
m_nChannels,
                             m_data.get(), m_width * m_nChannels) != 0;
}

void  Image::setColor(int  x,  int  y,  int  r,  int  g,  int  b)  {
     if (!m_data || x < 0 || y < 0 || x >= m_width || y >= m_height)
return;
       const  size_t  i  =  (static_cast<size_t>(y)  *  m_width  +  x)  *
m_nChannels;
                         m_data[i]      =      static_cast<uint8_t>(r);
                           if      (m_nChannels      >=      3)      {
                        m_data[i   +   1]   =   static_cast<uint8_t>(g);
                        m_data[i   +   2]   =   static_cast<uint8_t>(b);
                                                                      }
       // Alpha is not an argument of setColor and is therefore
preserved.
}

void Image::getColor(int x, int y, int& r, int& g, int& b) const {
     if (!m_data || x < 0 || y < 0 || x >= m_width || y >= m_height)
return;
       const  size_t  i  =  (static_cast<size_t>(y)  *  m_width  +  x)  *
m_nChannels;
                                       r         =         m_data[i];
      // For gray/gray-alpha the intensity is returned in r (as in
supplied                                                        tests).
           g   =   m_nChannels   >=   3   ?   m_data[i   +   1]   :   0;
           b   =   m_nChannels   >=   3   ?   m_data[i   +   2]   :   0;
}
```